Documento de Diseño

📐 Documento de Diseño Técnico

Sistema de Gestión Académica - Microservicios

---

1. Introducción

1.1 Propósito

Este documento describe el diseño técnico del Sistema de Gestión Académica, una aplicación basada en arquitectura de microservicios que permite gestionar estudiantes, cursos y matrículas.

1.2 Alcance

El sistema comprende cuatro microservicios independientes:

- **ms-estudiantes**: Gestión del ciclo de vida de estudiantes
- **ms-cursos**: Gestión del ciclo de vida de cursos
- **ms-matriculas**: Orquestación de matrículas con validaciones de negocio
- **api-gateway**: Punto de entrada único y enrutamiento

1.3 Definiciones y Acrónimos

Término	Definición
POO	Programación Orientada a Objetos
SOLID	Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
REST	Representational State Transfer
JPA	Java Persistence API
DTO	Data Transfer Object

---

2. Vista de Arquitectura

2.1 Diagrama de Contexto

```
┌──────────────
```

3. Diseño de Microservicios

3.1 MS-Estudiantes

3.1.1 Responsabilidades

- CRUD de estudiantes
- Gestión de estados (ACTIVE/INACTIVE)
- Validación de unicidad de email

3.1.2 Modelo de Dominio

```
┌──────────────────────────────────┐
│           Student                │
├──────────────────────────────────┤
│ - id: Long                       │
│ - firstName: String              │
│ - lastName: String               │
│ - email: String                  │
│ - status: StudentStatus          │
│ - createdAt: LocalDateTime       │
│ - updatedAt: LocalDateTime       │
├──────────────────────────────────┤
│ + activate(): void               │
│ + deactivate(): void             │
│ + isActive(): boolean            │
└──────────────────────────────────┘
         │
         │ <<enumeration>>
         ▼
┌──────────────────────────────────┐
│        StudentStatus             │
├──────────────────────────────────┤
│ ACTIVE                           │
│ INACTIVE                         │
└──────────────────────────────────┘
```

3.1.3 Capas

```
┌──────────────────────────────────────────────────┐
│                  Controller                      │
│            StudentController.java                │
│  - POST /students                                │
│  - GET /students/{id}                            │
│  - PATCH /students/{id}/status                   │
└──────────────────────────────────────────────────┘
                      │
                      │ DTO
                      ▼
```

```
┌──────────────────────────────────────────┐
│                  Service                 │
│            StudentService.java           │
│  - createStudent(dto): StudentResponse   │
│  - getStudentById(id): StudentResponse   │
│  - updateStatus(id, status): StudentResponse │
└──────────────────────────────────────────┘
                     │ Entity
                     ▼
┌──────────────────────────────────────────┐
│                Repository                │
│          StudentRepository.java          │
│  extends JpaRepository<Student, Long>    │
└──────────────────────────────────────────┘
                     │
                     ▼
┌──────────────────────────────────────────┐
│                 Database                 │
│               H2 (students)              │
└──────────────────────────────────────────┘
```

3.2 MS-Cursos

3.2.1 Responsabilidades

- CRUD de cursos
- Gestión de estados (ACTIVE/INACTIVE)
- Control de capacidad y cupos disponibles

3.2.2 Modelo de Dominio

```
┌──────────────────────────────┐
│           Course             │
├──────────────────────────────┤
│ - id: Long                   │
│ - name: String               │
│ - description: String        │
│ - capacity: Integer          │
│ - enrolledCount: Integer     │
│ - status: CourseStatus       │
│ - createdAt: LocalDateTime   │
│ - updatedAt: LocalDateTime   │
├──────────────────────────────┤
│ + hasAvailableCapacity(): bool │
│ + incrementEnrolled(): void  │
│ + decrementEnrolled(): void  │
│ + getAvailableSpots(): int   │
└──────────────────────────────┘
          │
```

```
          │ <<enumeration>>
          ▼

┌──────────────────────────────┐
│         CourseStatus         │
├──────────────────────────────┤
│ ACTIVE                       │
│ INACTIVE                     │
└──────────────────────────────┘
```

3.3 MS-Matrículas

3.3.1 Responsabilidades

- Orquestación de proceso de matrícula
- Validación de reglas de negocio
- Comunicación con otros microservicios

3.3.2 Modelo de Dominio

```
┌──────────────────────────────┐
│          Enrollment          │
├──────────────────────────────┤
│ - id: Long                   │
│ - studentId: Long            │
│ - courseId: Long             │
│ - enrollmentDate: LocalDateTime │
│ - status: EnrollmentStatus   │
├──────────────────────────────┤
│ + cancel(): void             │
│ + isActive(): boolean        │
└──────────────────────────────┘
          │
          │ <<enumeration>>
          ▼

┌──────────────────────────────┐
│       EnrollmentStatus       │
├──────────────────────────────┤
│ ACTIVE                       │
│ CANCELLED                    │
└──────────────────────────────┘
```

3.3.3 Reglas de Negocio

#	Regla	Código Error	HTTP Status
1	El estudiante debe existir	STUDENT_NOT_FOUND	404
2	El estudiante debe estar activo	STUDENT_INACTIVE	422
3	El curso debe existir	COURSE_NOT_FOUND	404

#	Regla	Código Error	HTTP Status
4	El curso debe estar activo	COURSE_INACTIVE	422
5	El curso debe tener cupo	COURSE_FULL	422
6	No debe existir matrícula duplicada	ENROLLMENT_CONFLICT	409

4. Patrones de Diseño

4.1 Strategy Pattern - Validaciones

```
                    ┌──────────────────────────────────┐
                    │ <<interface>>                    │
                    │ EnrollmentValidationStrategy     │
                    ├──────────────────────────────────┤
                    │ + validate(request): void        │
                    │ + getOrder(): int                │
                    └──────────────────────────────────┘
                                   △
                                   │
                    ┌──────────────┼──────────────┐
                    │              │              │
                    ▼              ▼              ▼
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
│StudentExistsValid│   │CourseActiveValid │   │CapacityValidation│
├──────────────────┤   ├──────────────────┤   ├──────────────────┤
│ - restClient     │   │ - restClient     │   │ - restClient     │
├──────────────────┤   ├──────────────────┤   ├──────────────────┤
│ + validate()     │   │ + validate()     │   │ + validate()     │
│ + getOrder(): 1  │   │ + getOrder(): 3  │   │ + getOrder(): 5  │
└──────────────────┘   └──────────────────┘   └──────────────────┘
```

Ventajas:

- Fácil agregar nuevas validaciones (OCP)
- Cada validación es independiente (SRP)
- Las validaciones son intercambiables (LSP)

4.2 Template Method Pattern - Pipeline de Matrícula

```
┌──────────────────────────────────────────────────────────┐
│           EnrollmentProcessTemplate                      │
│                 <<abstract>>                             │
├──────────────────────────────────────────────────────────┤
│ + processEnrollment(request): Enrollment  [final]        │
│ # validateRequest(request): void                         │
│ # validateBusinessRules(request): void [abstract]        │
│ # createEnrollment(request): Enrollment                  │
```

```
│ # postProcessing(enrollment): void [hook]          │
└────────────────────────────────────────────────────┘

                          △
                          │
              ┌───────────┴───────────┐
              │                       │
              ▼                       ▼

┌──────────────────────┐    ┌──────────────────────┐
│StandardEnrollmentProc│    │PriorityEnrollmentProc│
├──────────────────────┤    ├──────────────────────┤
│ # validateBusiness() │    │ # validateBusiness() │
│ # postProcessing()   │    │ # postProcessing()   │
└──────────────────────┘    └──────────────────────┘
```

Flujo del Template Method:

```
processEnrollment(request)
         │
         ▼

┌──────────────────────┐
│ 1. validateRequest   │  Validación de campos (Jakarta)
└──────────────────────┘
         │
         ▼

┌──────────────────────┐
│ 2. validateBusiness  │  Validación de reglas (Strategy)
│     [ABSTRACT]       │
└──────────────────────┘
         │
         ▼

┌──────────────────────┐
│ 3. createEnrollment  │  Persistir matrícula
└──────────────────────┘
         │
         ▼

┌──────────────────────┐
│ 4. postProcessing    │  Acciones post-creación (Hook)
│      [HOOK]          │
└──────────────────────┘
         │
         ▼
       return
```

4.3 Repository Pattern

Implementado a través de Spring Data JPA:

```java
@Repository
public interface EnrollmentRepository extends
        JpaRepository<Enrollment, Long> {

    List<Enrollment> findByStudentId(Long studentId);

    List<Enrollment> findByCourseId(Long courseId);
```

```
    boolean existsByStudentIdAndCourseIdAndStatus(
        Long studentId,
        Long courseId,
        EnrollmentStatus status
    );
}
```

---

5. Comunicación entre Servicios

5.1 Diagrama de Secuencia - Crear Matrícula

```
┌─────────┐      ┌────────────┐      ┌──────────────┐      ┌──────────┐
┌─────────┐
│ Cliente│      │API Gateway│      │MS-Matricula│      │MS-Student│
│MS-Curso│
└────┬────┘      └─────┬──────┘      └──────┬───────┘      └────┬─────┘
└────┬────┘
     │                 │                    │                   │
│
     │ POST /enrollments                    │                   │
│
     │ {studentId, courseId}                │                   │
│
     │────────────────►│                    │                   │
│
     │                 │                    │                   │
│
     │                 │ Forward + X-Correlation-Id             │
│
     │                 │───────────────────►│                   │
│
     │                 │                    │                   │
│
     │                 │                    │ GET /students/{id}
│
     │                 │                    │──────────────────►│
│
     │                 │                    │                   │
│
     │                 │                    │    Student data   │
│
     │                 │                    │◄──────────────────│
│
     │                 │                    │                   │
│
     │                 │                    │ GET /courses/{id}
│
     │                 │
```

```
│──────────────
```

```
            .defaultHeader("Content-Type", "application/json")
            .build();
    }

    @Bean
    public RestClient courseServiceClient() {
        return RestClient.builder()
            .baseUrl("http://localhost:8082/api/v1")
            .defaultHeader("Content-Type", "application/json")
            .build();
    }
}
```

---

6. Manejo de Errores

6.1 Estructura de Error Response

```
public record ErrorResponse(
    LocalDateTime timestamp,
    String path,
    String errorCode,
    String message,
    List<String> details
) {
    public static ErrorResponse of(String path, String errorCode,
                                   String message, List<String>
        details) {
        return new ErrorResponse(
            LocalDateTime.now(),
            path,
            errorCode,
            message,
            details
        );
    }
}
```

6.2 Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(ResourceNotFoundException ex,
                                        HttpServletRequest request) {
        return ErrorResponse.of(
            request.getRequestURI(),
```

```
            ex.getErrorCode(),
            ex.getMessage(),
            ex.getDetails()
        );
    }

    @ExceptionHandler(BusinessException.class)
    @ResponseStatus(HttpStatus.UNPROCESSABLE_ENTITY)
    public ErrorResponse handleBusinessException(BusinessException ex,
                                                 HttpServletRequest
        request) {
        return ErrorResponse.of(
            request.getRequestURI(),
            ex.getErrorCode(),
            ex.getMessage(),
            ex.getDetails()
        );
    }

    @ExceptionHandler(ConflictException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public ErrorResponse handleConflict(ConflictException ex,
                                        HttpServletRequest request) {
        return ErrorResponse.of(
            request.getRequestURI(),
            ex.getErrorCode(),
            ex.getMessage(),
            ex.getDetails()
        );
    }
}
```

6.3 Jerarquía de Excepciones

```
                    ┌──────────────────┐
                    │ RuntimeException │
                    └──────────────────┘
                            │
                            ▼
                    ┌──────────────────┐
                    │ BaseException    │
                    ├──────────────────┤
                    │ - errorCode: String│
                    │ - details: List  │
                    └──────────────────┘
                            │
                            │
        ┌───────────────────┼───────────────────┐
        │                   │                   │
        ▼                   ▼                   ▼
┌──────────────────┐ ┌──────────────────┐ ┌──────────────────┐
│ResourceNotFound  │ │BusinessException │ │ConflictException │
│   Exception      │ │                  │ │                  │
```

```
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│ HTTP 404         │  │ HTTP 422         │  │ HTTP 409         │
└──────────────────┘  └──────────────────┘  └──────────────────┘
```

---

7. Observabilidad

7.1 Correlation ID Filter

```java
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String CORRELATION_ID_HEADER = "X-Correlation-
        Id";

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) {
        String correlationId =
        request.getHeader(CORRELATION_ID_HEADER);

        if (correlationId == null || correlationId.isBlank()) {
            correlationId = UUID.randomUUID().toString();
        }

        MDC.put("correlationId", correlationId);
        response.setHeader(CORRELATION_ID_HEADER, correlationId);

        try {
            filterChain.doFilter(request, response);
        } finally {
            MDC.remove("correlationId");
        }
    }
}
```

7.2 Logging Pattern

```yaml
# application.yml
logging:
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%X{correlationId}] %-5level
        %logger{36} - %msg%n"
```

---

8. Aplicación de Principios SOLID

8.1 Single Responsibility Principle (SRP)

Clase	Única Responsabilidad
`StudentController`	Manejar requests HTTP de estudiantes
`StudentService`	Lógica de negocio de estudiantes
`StudentRepository`	Acceso a datos de estudiantes
`StudentMapper`	Conversión entre Entity y DTO
`StudentExistsValidation`	Validar existencia de estudiante

8.2 Open/Closed Principle (OCP)

Para agregar una nueva validación de matrícula:

1. Crear nueva clase implementando `EnrollmentValidationStrategy`
2. Anotar con `@Component`
3. El sistema la detecta automáticamente

```
@Component
public class NewBusinessRuleValidation implements
        EnrollmentValidationStrategy {
    @Override
    public void validate(EnrollmentRequest request) {
        // Nueva regla de negocio
    }

    @Override
    public int getOrder() {
        return 10; // Orden de ejecución
    }
}
```

8.3 Liskov Substitution Principle (LSP)

Todas las implementaciones de `EnrollmentValidationStrategy` pueden sustituirse entre sí sin afectar el comportamiento del sistema.

8.4 Interface Segregation Principle (ISP)

```
// Interfaces pequeñas y específicas
public interface StudentReader {
    StudentResponse findById(Long id);
}

public interface StudentWriter {
    StudentResponse create(StudentRequest request);
    StudentResponse updateStatus(Long id, StatusRequest request);
```

```
}

// El servicio implementa ambas
public interface StudentService extends StudentReader, StudentWriter {
        }
```

8.5 Dependency Inversion Principle (DIP)

- Las clases de alto nivel (Services) dependen de abstracciones (Repository interfaces)
- Inyección de dependencias vía constructor con Spring

```
@Service
@RequiredArgsConstructor
public class EnrollmentServiceImpl implements EnrollmentService {

    private final EnrollmentRepository repository;  // Abstracción
    private final List<EnrollmentValidationStrategy> validations; //
        Abstracciones

    // No depende de implementaciones concretas
}
```

---

9. Diagrama de Clases Completo

Ver archivo: `docs/diagrams/class-diagram.puml`

---

10. Consideraciones de Escalabilidad

10.1 Actuales

- Bases de datos H2 embebidas (desarrollo)
- Comunicación REST síncrona

10.2 Futuras Mejoras

- Migración a PostgreSQL (producción)
- Implementar Circuit Breaker (Resilience4j)
- Comunicación asíncrona con eventos (Kafka)
- Service Discovery (Eureka)
- Configuración centralizada (Spring Cloud Config)
- Containerización con Docker
- Orquestación con Kubernetes

Documento de Diseño v1.0